

Diseño de Tema — Módulo II: Pruebas Unitarias

Laboratorio de Programación y Lenguajes (IF009) — UNTDF 2026

Tema: 02 — Módulo II: Pruebas Unitarias

Semanas: 4–5 (12 hs: 2 sesiones teóricas + 2 prácticas de 3 hs c/u)

Estado: En diseño

Docente: Matías Gel

Agente diseñador: Lic. Marcos (topic-designer)

Generado: 2026-04-15

1. Ancla Curricular

Plan mínimo (inmutable):

- Importancia de la detección oportuna de errores
- Clasificación y tipos de pruebas
- Prueba de Unidad: definición y propósito
- Procedimiento para pruebas de unidad
- Soporte del lenguaje Python para pruebas de unidad (unittest)

Plan borrador (propuestas integradas):

- **P4** — TDD mandatorio desde Módulo II
- **P5** — AI pair programming (Copilot en Codespaces)

Continuidad con Módulo I: Los alumnos ya conocen Python básico (tipos, funciones, colecciones, lambdas, type hints). El Módulo II **comienza con un repaso de POO en Python** como base para entender TestCase como clase y para el ejercicio integrador (clase Agenda).

2. Objetivos de Aprendizaje

Al finalizar este módulo el estudiante podrá:

- 1. **Recordar** los conceptos fundamentales de POO en Python (clases, instancias, herencia, excepciones)
- 2. **Comprender** la pirámide de testing y la diferencia entre pruebas de caja negra y caja blanca
- 3. **Aplicar** el framework `unittest` para escribir tests unitarios con cobertura de casos normales y de borde
- 4. **Aplicar** el ciclo TDD (Red → Green → Refactor) sobre una clase con lógica de negocio y excepciones
- 5. **Analizar** qué casos de prueba son necesarios para cubrir un módulo dado (criterio de completitud)
- 6. **Construir** tests con fixtures (`setUp` , `tearDown` , `setUpClass`) y aserciones de excepciones (`assertRaises`)

Taxonomía Bloom: Recordar (1) → Comprender (2) → Aplicar (3, 4) → Analizar (5) → Crear (6)

3. Mapa de Contenidos

Bloque 0 — Repaso POO Python (apertura, 30 min, semana 4 teoría)

Justificación pedagógica: `TestCase` es una clase que se hereda. El ejercicio integrador (Agenda) es una clase con excepciones. El alumno necesita tener fresco el modelo de objetos antes de ver `unittest`.

Subtema	Profundidad	Fuente
Clases e instancias: <code>__init__</code> , atributos, métodos	Repaso rápido	Módulo I
Herencia: <code>class Hijo(Padre)</code> , <code>super()</code>	Repaso rápido	Módulo I
Métodos especiales: <code>__str__</code> , <code>__repr__</code> , <code>__eq__</code>	Nuevo (necesario para <code>assertEqual</code>)	Módulo II
Excepciones propias: <code>class MiError(Exception)</code>	Nuevo	Módulo II
Type hints en clases: atributos tipados, <code>Optional</code>	Repaso	Módulo I

Ejemplo disparador:

```
class Agenda:
    def __init__(self, nombre: str, dni: str) -> None:
        if not dni.isdigit():
            raise ValueError(f"DNI inválido: {dni}")
        self.nombre = nombre
        self.dni = dni
```

“¿Cómo verificamos que esto funciona — y que el error se dispara cuando debe?”

Bloque 1 — Motivación: Por qué testar (40 min, semana 4 teoría)

Énfasis del docente: arrancar fuerte con los costos reales del software sin tests. Las filminas 2025 tienen el material base — se amplía con clasificación completa de tipos de tests.

1a. El costo de NO testear (filminas 2025 — slide “Errores!!!”)

Caso	Problema	Consecuencia	Fuente
Vuelo Air China 140	Error de software desactivó el piloto automático	264 muertes	✓ python-testing.pdf
Therac-25	Overflow de software: emitía 100× más radiación de lo normal bajo ciertas condiciones de interfaz	4 muertes	✓ python-testing.pdf
Ariane 5 (1996)	Overflow al reutilizar software de Ariane 4 sin re-testear — desviación de trayectoria	Pérdidas millonarias, destrucción del cohete	✓ python-testing.pdf

Pregunta disparadora para el aula: “¿Qué tienen en común estos tres casos?” → todos podían haberse detectado con tests de regresión o pruebas de borde antes del despliegue.

Slide de apoyo: python-testing.pdf — slide “Testing inadecuado” cita: “Los errores en el software no identificados a tiempo pueden ser sumamente costosos en términos monetarios o incluso llevar a pérdidas de vidas humanas.”

1b. Qué es el testing de software (filminas 2025 — slide “Testing de software”)

Definición directa de las filminas: “Es un proceso que busca verificar la exactitud, integridad y calidad de un software. Incluye una serie de actividades que buscan encontrar errores y fallas en el software previo a que lleguen al usuario.”

1c. Tipos de tests — Clasificación por conocimiento (filminas 2025 — slide “Testing de software - niveles”)

Tipo	Cómo se diseñan las entradas	Ejemplo
Prueba de Caja Negra	Sin conocer el funcionamiento interno — se prueban entradas y salidas contra la especificación	Dar DNI inválido → esperar ValueError
Prueba de Caja Blanca	Conociendo la estructura interna — los datos se eligen para ejercitar ramas, condiciones y caminos específicos	Dar un DNI de exactamente 8 dígitos para cubrir el límite del <code>if len(dni)</code> < 7

1d. Niveles y tipos de tests — Pirámide de testing (ampliación 2026)

Nivel	Nombre	Qué prueba	Herramienta (stdlib)	Velocidad
1 (base)	Prueba de Unidad (Unit Test)	Una sola clase o función aislada	<code>unittest</code>	Milisegundos
2	Prueba de Integración	Interacción entre módulos o componentes	<code>unittest</code> + fixtures reales	Segundos
3	Prueba de Sistema	El sistema completo desde el exterior	<code>unittest</code> + <code>http.client</code> / CLI	Minutos
4 (cima)	Prueba de Aceptación	Cumplimiento de requisitos del usuario	Escenarios escritos por el docente/cliente	Minutos/horas

Foco de este módulo: Nivel 1 (Prueba de Unidad) usando `unittest`. Los niveles 2-4 se mencionan para dar contexto — los alumnos entenderán dónde “encaja” lo que están aprendiendo.

Definición de filminas 2025 (slide “Test de unidades”): “Son pruebas de bajo nivel (a nivel de código) que se focalizan en una pequeña parte del software. En programación orientada a objetos estas unidades suelen ser las clases.”

1e. Pruebas de regresión (concepto clave)

Una **prueba de regresión** es re-ejecutar los tests existentes después de un cambio. Si pasan, el cambio no “rompió” nada. Este concepto es central en TDD: el suite de tests es la red de seguridad para refactorizar.

Cierre del bloque: “En este módulo van a aprender a escribir el Nivel 1. Pero la pirámide existe porque el Nivel 1 solo no alcanza — y ustedes ya vieron por qué con los casos de Therac-25 y Ariane 5.”

Bloque 2 — Framework unittest (50 min, semana 4 teoría)

Fuente primaria: `python-unittest-reference.md` (ChromaDB) + `python-testing.pdf` (filminas 2025)

Subtema	Filminas 2025	Docs Python 3.14
Familia xUnit — historia (Kent Beck, Smalltalk, SUnit)	✓ python-testing.pdf	—
Conceptos: TestCase, TestSuite, TestRunner	✓ python-testing.pdf	Sección 1
Convención test_ — cómo el runner descubre tests	✓ python-testing.pdf — “La clase TestCase”	Sección 2
Aserciones esenciales: assertEquals , assertTrue , assertFalse , assertIsNone	✓ python-testing.pdf	Sección 4 tabla básica
Aserciones de excepciones: assertRaises con with	✓ python-testing.pdf — “assertRaises verifica...”	Sección 4 tabla excepciones
Fixtures: setUp , tearDown , setUpClass , tearDownClass	✓ python-testing.pdf — “La clase TestCase - fixtures”	Sección 3
Ejecución CLI: python -m unittest -v , descubrimiento automático	✓ python-testing.pdf — “Ejecución de test de línea de comandos”	Sección 7-8

Ejemplo de clase — clase TestCase mínima:

```
import unittest
from agenda import Agenda

class AgendaTest(unittest.TestCase):

    def setUp(self):
        self.ag = Agenda("Juan", "12345678")

    def test_nombre_correcto(self):
        self.assertEqual(self.ag.nombre, "Juan")

    def test_dni_invalido_lanza_excepcion(self):
        with self.assertRaises(ValueError):
            Agenda("Juan", "abc")

if __name__ == '__main__':
    unittest.main()
```

Bloque 3 — TDD: Red → Green → Refactor (30 min, semana 4 teoría)

Subtema	Notas
Origen TDD — Kent Beck, XP	Vincula con historia xUnit del bloque 2
Ciclo Red-Green-Refactor: escribir test primero, verlo fallar, implementar mínimo, limpiar	Central — repetir con demostración en vivo
Beneficios: diseño emergente, documentación ejecutable, confianza al refactorizar	
Demo en vivo: FizzBuzz desde el test	Kata clásica — 3 iteraciones del ciclo completo

Demo FizzBuzz TDD:

```
Red: test_fizz_retorna_fizz → NameError (función no existe)
Green: def fizzbuzz(n): return "Fizz" → AssertionError en otros tests
Green: implementación mínima completa
Refactor: type hints, docstring, extraer constantes
```

Bloque 4 — Ejercicio integrador: clase Agenda (semana 4 práctica + continuación semana 5)

Fuente directa filminas 2025 — [python-testing.pdf](#) — slide “Problema”: “Realizar la clase Agenda que guarde nombre, apellido, dirección y teléfono de una persona utilizando su DNI como referencia. Desarrolle los métodos necesarios para gestionar los datos incluyendo excepciones por datos inválidos. Una vez realizada la clase Agenda escriba la clase AgendaTest que pruebe todos los métodos incluidas las excepciones.”

Variante 2026 (ampliada): Se aplica TDD — los tests se escriben **antes** de la clase Agenda.

Especificación:

Método	Comportamiento esperado	Test(s) requerido(s)
<code>__init__(dni, nombre, apellido)</code>	Valida DNI numérico 7-8 dígitos; lanza <code>ValueError</code> si inválido	<code>test_crear_valido</code> , <code>test_dni_no_numerico</code> , <code>test_dni_muy_corto</code>
<code>agregar(dni, nombre, apellido, direccion, telefono)</code>	Registra la persona; lanza <code>KeyError</code> si DNI ya existe	<code>test_agregar_nuevo</code> , <code>test_agregar_duplicado</code>
<code>buscar(dni)</code>	Retorna dict con datos; lanza <code>KeyError</code> si no existe	<code>test_buscar_existente</code> , <code>test_buscar_inexistente</code>
<code>eliminar(dni)</code>	Elimina registro; lanza <code>KeyError</code> si no existe	<code>test_eliminar_existente</code> , <code>test_eliminar_inexistente</code>
<code>listar()</code>	Retorna lista de todos los DNIs registrados	<code>test_listar_vacia</code> , <code>test_listar_con_datos</code>

Estructura de fixtures para el test:

```
class AgendaTest(unittest.TestCase):

    def setUp(self):
        self.agenda = Agenda()
        self.agenda.agregar("12345678", "Juan", "Pérez", "Calle 1", "2901-111")
        self.agenda.agregar("87654321", "Ana", "García", "Calle 2", "2901-222")

    def tearDown(self):
        del self.agenda
```

Bloque 5 — `unittest.mock` y CI (*semana 5 teoría + práctica*)

Restricción del docente: solo se usa `unittest` de la stdlib. No se enseña pytest. El mocking se cubre únicamente con `unittest.mock` (módulo oficial de Python, parte de la stdlib).

Subtema	Profundidad
<code>unittest.mock.MagicMock</code> : crear dobles de objetos	Conceptual + ejemplo concreto
<code>unittest.mock.patch</code> como context manager y decorador	Cuándo aislar dependencias externas
Ejemplo: testear función que llama a <code>open()</code> o a una conexión de BD	Demo en vivo
<code>python -m unittest discover</code> — descubrimiento automático en proyecto real	Flujo de trabajo completo
GitHub Actions: autograding con <code>python -m unittest</code> (ya visto en TP 2)	Refuerzo — el alumno ya lo vio en práctica

Nota sobre pytest: se menciona en una sola diapositiva como herramienta que existe en el ecosistema, sin profundizar ni practicarlo.

Bloque 6 — Integración con BlogApp (20 min, semana 5 teoría)

Subtema	Notas
Ubicación de tests en el proyecto Django: carpeta <code>tests/</code>	Estructura recomendada (ChromaDB sección 15)
Anticipo <code>django.test.TestCase</code> — subclase de <code>unittest.TestCase</code> (se profundiza en Módulo III)	Todo lo aprendido en este módulo aplica directamente
TP 3: estructura con tests incluidos desde el inicio	Link directo al trabajo práctico

4. Estructura de Sesiones

Semana 4 — Sesión Teórica (3 hs = 180 min)

Bloque	Duración	Contenido	Metodología
T0	30 min	Repaso POO Python: clases, herencia, excepciones propias, <code>__str__</code>	Repaso activo: preguntas al grupo, ejemplo Agenda disparador
T1	40 min	Motivación: errores históricos (Air China, Therac-25, Ariane 5), costo del NO-testing, tipos de tests (caja negra/blanca, pirámide unit→integración→sistema→aceptación), pruebas de regresión	Storytelling con filminas 2025 + pirámide de niveles
T2	40 min	Framework <code>unittest</code> : <code>TestCase</code> , aserciones, fixtures, <code>assertRaises</code> , CLI	Exposición + código en vivo
—	10 min	Pausa	
T3	30 min	TDD: ciclo Red-Green-Refactor, FizzBuzz demo en vivo	Demostración iterativa con 3 ciclos completos
T4	20 min	Cierre: preview ejercicio Agenda + setup GitHub Classroom TP 3	

Total: 160 min de contenido + 10 min pausa + 10 min buffer = 180 min 

Semana 4 — Sesión Práctica (3 hs = 180 min)

Bloque	Duración	Contenido	Metodología
P1	60 min	Kata TDD guiada: FizzBuzz desde test en Codespaces	Pair programming — alumnos en parejas
P2	60 min	Ejercicio Agenda — fase 1: escribir los tests de <code>__init__</code> , <code>agregar</code> y <code>buscar</code> antes de implementar la clase	TDD individual con soporte docente
P3	40 min	Ejercicio Agenda — fase 2: implementar la clase Agenda hasta pasar los tests escritos	
P4	20 min	Revisión colectiva: mostrar tests fallando → implementar → verde	Live coding al frente

Semana 5 — Sesión Teórica (3 hs = 180 min)

Bloque	Duración	Contenido	Metodología
T1	45 min	<code>unittest.mock</code> : <code>MagicMock</code> , <code>patch</code> como context manager y decorador. Cuándo aislar dependencias	Exposición + demo: testear función que llama a <code>open()</code> o BD
T2	30 min	Subtests y skipping: <code>subTest()</code> , <code>@skip</code> , <code>@skipIf</code> , <code>@expectedFailure</code>	Casos de uso reales
—	10 min	Pausa	
T3	30 min	TDD aplicado a BlogApp: <code>django.test.TestCase</code> hereda de <code>unittest.TestCase</code> (anticipo)	Demo conceptual — muestra continuidad directa
T4	30 min	Cierre Módulo II: repaso completo, dudas, presentación TP 3	
T5	25 min	Revisión TP 2 (feedback grupal de los tests de autograding)	

Total: 160 min + 10 min pausa + 10 min buffer = 180 min 

Semana 5 — Sesión Práctica (3 hs = 180 min)

Bloque	Duración	Contenido	Metodología
P1	60 min	Taller: escribir tests para los scripts del TP 2 que no tienen tests aún	Individual con Copilot habilitado
P2	60 min	GitHub Classroom TP 3: clonar repo, explorar estructura Django básica, correr suite de tests inicial	Guiado
P3	60 min	Completar ejercicio Agenda: métodos <code>eliminar</code> y <code>listar</code> con sus tests	TDD

5. Ejercicios Clave

E1 — Kata FizzBuzz TDD (guiada)

- **Nivel:** Introductorio
- **Objetivo:** Vivir el ciclo Red-Green-Refactor por primera vez
- **Tiempo:** 60 min (práctica semana 4)
- **Trazabilidad:** Bloque T3 (TDD)

E2 — Clase Agenda (integrador del módulo)

- **Nivel:** Intermedio
- **Objetivo:** Diseñar tests antes de implementar una clase con excepciones y CRUD interno
- **Tiempo:** 80 min distribuidos en semana 4 práctica + 60 min semana 5 práctica
- **Origen:** Filminas 2025 (`python-testing.pdf` — slide “Problema”)
- **Ampliación 2026:** Aplicar TDD estricto + añadir `eliminar` y `listar`
- **Trazabilidad:** Bloques T2 (fixtures, assertRaises), T3 (TDD)

E3 — Tests para TP 2 (práctica real)

- **Nivel:** Aplicación directa
 - **Objetivo:** Escribir tests para código propio ya escrito — inversión del flujo TDD clásico
 - **Tiempo:** 60 min (práctica semana 5)
 - **Trazabilidad:** Cierre Módulo II
-

6. Materiales de Referencia

Material	Tipo	Fuente
<code>python-testing.pdf</code> (filminas 2025)	Presentación original del docente	<code>ingesta/txt/python-testing.txt</code> (ChromaDB: material)
<code>python-unittest-reference.md</code>	Referencia técnica Python 3.14	<code>_edu-knowledge/tools/</code> (ChromaDB: tool)
<code>testing.md</code>	Resumen para alumnos (Módulo II)	<code>salida/cursadas/2026/temas/02-pruebas-unitarias/testing.md</code>
Docs oficiales <code>unittest</code>	Referencia primaria	https://docs.python.org/3/library/unittest.html
Docs oficiales <code>unittest.mock</code>	Referencia primaria	https://docs.python.org/3/library/unittest.mock.html

7. Prerequisitos del Módulo

Prerequisito	Fuente	Estado
Python: variables, funciones, colecciones	Módulo I	✅ cubierto
Python: clases básicas, <code>__init__</code> , métodos	Módulo I	✅ cubierto
Python: excepciones (<code>try/except/raise</code>)	Módulo I	✅ cubierto
Python: herencia, <code>super()</code>	Módulo I	⚠️ repaso necesario (Bloque T0)
GitHub Classroom + Codespaces	Módulo 0/I	✅ cubierto
type hints básicos	Módulo I	✅ cubierto

8. Alineación con Plan Borrador

Ítem plan-borrador	Cobertura en este diseño
Detección oportuna de errores	✓ Bloque 1 (casos Therac-25, Ariane 5, Air China)
Taxonomía de pruebas / Pirámide	✓ Bloque 1 (niveles unit/integración/sistema/aceptación + caja negra/blanca + regresión)
Prueba de Unidad: ciclo Red-Green-Refactor	✓ Bloque 3
<code>assert</code> semánticos	✓ Bloque 2 (tabla completa de aserciones)
<code>unittest</code> fixtures, subtests, skipping	✓ Bloques 2 + 5
TDD en la práctica	✓ Bloques 3 + 4 + ejercicios
Kata TDD FizzBuzz	✓ E1
<code>python -m unittest discover</code> (CI)	✓ Bloque 5, semana 5
GitHub Actions (CI)	✓ Bloque 5 (refuerzo)
Mocking: <code>unittest.mock</code> , <code>MagicMock</code> , <code>patch</code> (stdlib oficial)	✓ Bloque 5, semana 5
<code>django.test.TestCase</code> (anticipo, hereda de <code>unittest.TestCase</code>)	✓ Bloque 6
TDD aplicado a BlogApp	✓ Bloque 6

9. Métricas de Carga Cognitiva (guardrail)

Dimensión	Valor	Límite recomendado
Conceptos nuevos por sesión teórica	~6 (T0 repaso + 5 nuevos)	≤ 7
Duración sesión teórica	180 min	180 min
Ejercicios por sesión práctica	2 + cierre	≤ 3
Nivel Bloom máximo en TP	Crear (E2 — diseño de tests desde cero)	OK para módulo 2

10. Pendientes / Preguntas para el Docente

- ☐ [] **Repaso POO (T0):** ¿Cuánto se cubrió realmente en Módulo I? ¿Vieron herencia o solo clases álicas?
 - ☐ [] **Agenda:** ¿Se usa esta misma clase en el TP 3 (Django), o se diseña una nueva? (impacta si conviene acerla persistente con dict o lista)
 - ☐ [] **Mocking en semana 5:** ¿Querés introducir `patch` como decorador o solo como context manager n semana 5?
 - ☐ [] **Coverage:** ¿Se mide cobertura con `coverage.py` (`python -m coverage run -m unittest discover`) o o se pide en este módulo?
-

Diseño generado por: Lic. Marcos (topic-designer) — 15/04/2026

Fuentes consultadas: `python-testing.pdf` (filminas 2025, ChromaDB), `python-unittest-reference.md` (docs Python 3.14, ChromaDB), `plan-borrador.md` , `plan-minimo.md`